

Pengu Engine

Subject: Graphics Engine Programming
Course: 3º VGP
Teacher: Arnau
Year: 2025/2026
Authors: Kaelin J. Lind



Index

1. Introduccion.....	3
2. Engine Features.....	4
3. Implemented Techniques.....	5
Physically Based Rendering (PBR).....	5
Deferred Rendering Pipeline.....	6
Shadow Mapping (Forward Shadows).....	7
Screen Space Reflections (SSR).....	7
4. Strategies and Solutions to Development Problems.....	9
5. Tasks Distribution Among Developers.....	10



1. Introduccion

This documentation provides a comprehensive breakdown of Pengu Engine, a custom C++ game engine I developed over the 2025/26 academic year. It details the core architecture, including the Sparse Set ECS and the deferred rendering pipeline, while examining the technical hurdles I faced and the engineering strategies I used to resolve them.



2. Engine Features

Pengu Engine is a data-oriented, high-performance game engine built in C++17. Its core architecture revolves around flexibility and fast iteration:

- **Data-Oriented Architecture:** The engine utilizes an Entity Component System (ECS) built on a Sparse Set architecture. This ensures cache coherency, contiguous memory allocation for components, and lightning-fast iteration over active entities.
- **Job System:** A custom job system allows for parallel processing of tasks to maximize CPU utilization.
- **Resource Management:** A centralized pipeline handles the loading, caching, and automated cleanup of meshes, textures, materials, and shaders.
- **Lua Scripting with Hot-Reload:** Gameplay logic is integrated via Lua. Scripts support runtime hot-reloading, enabling rapid iteration without needing to recompile the engine. The API exposes entity manipulation, component access, and standardized lifecycle hooks ([onStart](#), [onUpdate](#), [onFinish](#)).
- **Cross-Platform Windowing & Input:** Built on GLFW, providing abstracted input handling that supports both direct polling and action-based bindings.
- **Editor & Debugging Tooling:** Deep ImGui integration provides real-time scene inspection, entity management, and rendering pipeline diagnostics directly in the engine UI.
- **Terrain Generation:** The engine includes a procedural terrain generator that dynamically constructs landscapes. It utilizes noise algorithms to mathematically generate heightmaps, which are then applied to subdivide and displace plane geometry at runtime, creating seamless topological features.



3. Implemented Techniques

Pengu Engine supports a dynamic rendering pipeline backend using OpenGL. Pipelines can be swapped at runtime, including **Unlit**, **Forward**, **Forward Shadows**, **Deferred Rendering**, and a full **PBR (Physically Based Rendering)** shading model.

Below is an in-depth tutorial breakdown of the advanced rendering techniques implemented to demonstrate technical mastery.

Physically Based Rendering (PBR)

To achieve photorealistic lighting and material interactions, Pengu Engine implements a PBR shading model based on the Cook-Torrance BRDF (Bidirectional Reflectance Distribution Function). This replaces traditional Phong/Blinn-Phong models with physically accurate math.

How it works:

PBR relies on three core principles: microfacet theory, energy conservation, and a physically based BRDF.

1. **Material Inputs (The PBR Workflow):** Instead of arbitrary specular highlights, PBR uses specific texture maps: **Albedo** (base color), **Normal** (surface detail), **Metallic** (conductive vs. dielectric), **Roughness** (microscopic surface variations), and **AO** (Ambient Occlusion).
2. **Energy Conservation:** We calculate light mathematically to ensure that the amount of light reflected off a surface never exceeds the amount of light hitting it. The light is split into a specular part (reflected light) and a diffuse part (refracted light that scatters inside the material and exits). Metallic surfaces only have a specular response, while dielectrics have both.
3. **The Cook-Torrance BRDF Math:** In the fragment shader, we compute the specular reflection using three functions:
 - **Normal Distribution Function (NDF):** We use *Trowbridge-Reitz GGX* to calculate how many microscopic bumps (microfacets) are aligned exactly halfway between the light source and the camera.



Rougher materials have more scattered microfacets, leading to larger, dimmer highlights.

- **Geometry Function:** We use *Schlick-GGX* to calculate microfacet self-shadowing. At grazing angles, microscopic bumps cast shadows on each other, reducing the light that reaches the camera.
 - **Fresnel Equation:** We use the *Fresnel-Schlick approximation* to calculate the ratio of light that is reflected versus refracted based on the viewing angle. Surfaces become highly reflective at grazing angles.
4. **Final Output:** We combine these three functions ($D * F * G$) divided by a normalization factor to get the final specular light, add it to the Lambertian diffuse calculation, and multiply by the incoming radiance to get the final pixel color.

Deferred Rendering Pipeline

Traditional forward rendering calculates lighting for every object as it is drawn, which becomes highly inefficient if many fragments are eventually occluded by other objects or if the scene has many lights. To solve this, Pengu Engine implements a Deferred Rendering pipeline, separating the geometry rendering from the lighting calculations.

How it works:

Deferred rendering is achieved in two distinct passes:

1. **The Geometry Pass:** Instead of rendering the scene directly to the screen, we render the scene's geometric data into a collection of textures called a **G-Buffer** (Geometry Buffer). In Pengu Engine, this G-Buffer stores position/depth, normal vectors, and albedo/material data. Multiple Render Targets (MRTs) in OpenGL are used to write to these different texture buffers simultaneously.
2. **The Lighting Pass:** Once the G-Buffer is populated, we render a single 2D quad covering the screen. The fragment shader for this quad samples the data from the G-Buffer. Since we only execute this shader for the visible pixels on the



screen, we calculate the lighting equation only once per pixel, regardless of how many objects overlap in the 3D scene.

Shadow Mapping (Forward Shadows)

To ground objects in the world, the engine implements depth-dependent shadow mapping for both point and directional lights.

How it works:

Shadow mapping relies on the concept that if a surface is visible to the light source, it is illuminated; if something blocks the light's view, it is in shadow.

1. **The Depth Pass:** We set up a projection matrix from the exact position and orientation of the light. We render the scene, but disable color writing to only capture the depth buffer. This generates a **Shadow Map**—a texture where each pixel represents the distance from the light to the closest surface.
2. **The Camera Pass:** We render the scene from the player's camera. For every fragment, we calculate its position relative to the light space.
3. **The Depth Test:** We compare the fragment's calculated distance to the light against the value stored in the Shadow Map. If the fragment's depth is greater than the map's depth, another object is blocking the light, and the fragment is rendered in shadow.

Screen Space Reflections (SSR)

SSR is an advanced post-processing technique used to render real-time reflections on glossy or metallic surfaces. Instead of rendering the entire scene again from a mirrored perspective (which is incredibly expensive), SSR uses the data already present on the screen.

How it works:

1. **The Data Requirement:** SSR relies heavily on the Deferred Rendering G-Buffer. It requires the screen's Depth map, Normal map, and Color map.



2. **Calculating the Reflection Ray:** For any given pixel on screen, we calculate the vector from the camera to that pixel. We then reflect this view vector against the surface normal (sampled from our G-Buffer Normal map) to get our reflection ray.
3. **Ray Marching:** We convert this 3D reflected ray into 2D screen-space coordinates. We then "march" (step forward incrementally) along this 2D line across the screen.
4. **Depth Intersection Testing:** At each step of the march, we sample the Depth map at our current 2D coordinate. We compare this sampled depth against the actual depth of our traveling ray. If the ray's depth is greater than the sampled depth, our ray has "collided" with geometry on the screen.
5. **Sampling and Blending:** Once an intersection is found, we sample the Color map at that specific hit coordinate. This color is the reflection. We then blend this with the original pixel color, adding edge-fading to smoothly hide reflections when the ray travels off-screen where no data exists.



4. Strategies and Solutions to Development Problems

- **Problem: Cache Misses and Iteration Speed.** Early entity management designs suffered from poor memory locality, causing CPU cache misses when iterating through thousands of objects.
 - **Solution:** Implemented a Data-Oriented Sparse Set ECS. By storing components of the same type in tightly packed, contiguous arrays, the CPU can prefetch memory efficiently, drastically improving frame times.
- **Problem: Workflow Interruption.** Recompiling the C++ engine to test minor gameplay tweaks heavily slowed down the iteration cycle.
 - **Solution:** Integrated Lua scripting with runtime hot-reloading. Game logic can now be altered on the fly, allowing for immediate visual and mechanical feedback.



5.Tasks Distribution Among Developers

As this is an independently developed engine architecture, all modules were researched, designed, and implemented by a single developer:

- **Kaelin Joaquin Lind:**
 - Core Engine Architecture & Data-Oriented ECS
 - Procedural Terrain Generation Implementation
 - OpenGL Pipeline Abstraction (Unlit, Forward, Forward Shadows, Deferred)
 - Rendering Techniques Mathematics (Shadow Mapping, SSR, Light Accumulation)
 - Lua Scripting Integration & Hot-Reloading Infrastructure
 - ImGui Tooling & Editor UI Construction